

51 单片机实战记录

时间：2026-09-30 晚 ~ 2026-10-01 下午 **主题：**从"看过教程"到"自己写出第一个能跑的程序" **成果：**串口通信 + 按键计数程序完成并上传 GitHub

目录

- 一、起点：一个关键诊断
- 二、项目成果：串口 + 按键计数
- 三、踩坑全记录 (15 条)
- 四、原理补课
- 五、学习方法：输出型打法
- 六、水平定位与路径
- 七、工具与环境备忘
- 八、下一步计划

一、起点：一个关键诊断

我的学习状态（起点盘点）

学完 51 单片机教程，覆盖：定时器中断、串口、I2C、UART、SPI、CAN、DS18B20、矩阵按键、红外遥控。**显示屏 (LCD1602 / OLED) 没学。**

但四个"产出类"问题全部是"没有"：

问题	答案
独立做过的完整东西	没有
调试工具（万用表 / 示波器）	没有
读过一颗芯片的 datasheet	没有
代码上传过 GitHub	没有

诊断结论

问题不是知识不够，是"零产出"。

教程覆盖面已经很广（广度上够 L3 了），但实际水平停在 L2 偏下——因为知识全是从教程"流过去"的，**没有一样是自己生产出来的。**

这跟自述的"输出型学习者"完全吻合：**纯输入的东西留不住。**

因此定的路线

停止往下学新外设（CAN、SPI 再学一遍还是白学），改为把学过的东西重新生产出来：

步骤	内容	状态
第 1 步	按键 + 串口输出（原定矩阵按键，改用独立按键降低门槛）	✅ 今天完成
第 2 步	DS18B20 温度采集 + 串口上报	待做
第 3 步	加定时器中断做周期采样（非阻塞）	待做

关键决定：矩阵按键先放一放。原因是我当时分不清 16 个按键的两端分别接在 P1 的哪两个引脚上，也没有万用表可以测。换独立按键能立刻开工，先拿到"自己写的东西能跑"这个正反馈。

二、项目成果：串口 + 按键计数

2.1 功能

按一次独立按键，计数加一，通过串口以 **9600 波特率** 发送到电脑串口助手，一行一个数字。

```
1
2
3
...
10
11
```

2.2 硬件接线

功能	单片机引脚	对端	说明
独立按键	P3.2	GND	不按下 = 高，按下 = 低
串口发送	P3.1 (TXD)	USB 转串口模块的 RXD	必须交叉
串口接收	P3.0 (RXD)	USB 转串口模块的 TXD	必须交叉
晶振	—	11.0592 MHz	保证 9600 波特率整除

坑预警：绝对不能把按键接在 P3.0 / P3.1 上——那是串口的 RXD/TXD，占了它俩串口就废了。

2.3 完整代码

```
#include "reg52.h"

typedef unsigned char u8;
typedef unsigned int u16;

sbit KEY3 = P3^2;          // 独立按键
u8 count = 0;              // 计数

void delay_ten_us(u16 ten_us)
{
    while(ten_us--);
}

/* 串口发送一个字节（封装：避免那 4 行重复写多遍） */
void uart_send(u8 dat)
{
    TI = 0;                // 清发送完成标志
    SBUF = dat;            // 写入即启动发送
    while(!TI);            // 等待发送完成
    TI = 0;                // 手动清标志（硬件不会自动清）
}

void main()
{
    KEY3 = 1;              // 读之前先写 1（准双向口，必须）

    TMOD = 0x20;           // T1 方式 2（8 位自动重装）
    TH1 = 0xFD;            // 9600 波特率 @ 11.0592MHz
    TL1 = 0xFD;
    SCON = 0x50;           // 方式 1（8 位 UART）+ REN 允许接收
    PCON &= 0x7F;          // SMOD = 0，波特率不加倍
    TR1 = 1;               // 启动波特率发生器

    while(1)
    {
        if(KEY3 == 0)      // 读到 0 = 按下
        {
            delay_ten_us(5000); // 消抖延时
            if(KEY3 == 0)    // 二次确认
            {
                count++;
                uart_send(count / 10 + '0'); // 十位
            }
        }
    }
}
```

```
        uart_send(count % 10 + '0');    // 个位
        uart_send('\n');                // 换行
        while(KEY3 == 0);                // 等松手，防止按住狂加
    }
}
}
```

2.4 关键实现解析

(1) 串口初始化：5 个寄存器

寄存器	值	作用
TMOD	0x20	T1 方式 2（8 位自动重装）。波特率需要稳定的溢出率，所以必须用方式 2
TH1 TL1	0xFD	9600 波特率初值，两个都要写（TL1 计数、TH1 存重装值）
SCON	0x50	SM0=0，SM1=1 选方式 1；REN=1 打开接收
PCON	&= 0x7F	SMOD 清 0（波特率不加倍）。只动目标位是严谨写法
TR1	1	启动波特率发生器（最容易漏，漏了串口完全无输出）

(2) 波特率公式与 11.0592MHz 的由来

波特率 = $f_{osc} \div 12 \div (256 - TH1) \div 32 = f_{osc} \div (384 \times (256 - TH1))$

反推： $256 - TH1 = 11059200 \div (384 \times 9600) = 3$ ，所以 $TH1 = 253 = 0xFD$ 。

为什么晶振必须是 11.0592MHz：分母里 $384 = 12 \times 32$ 是固定的，要得到精确波特率， f_{osc} 必须能被 $384 \times \text{波特率}$ 整除。11.0592MHz 是专门挑出来的数——它整除 4800 / 9600 / 19200 等所有常见波特率。换成 12MHz，只能得到 10417 波特，误差 +8.5%，远超异步串口 $\pm 2\sim 3\%$ 的容差，结果就是满屏乱码。

(3) 按键消抖：三段式

```
if (KEY3 == 0)                // ① 检测到按下
{
    delay_ten_us(5000);        // ② 延时（约 45ms）等抖动结束
    if (KEY3 == 0)              // ③ 再确认一次还是按下
    { ... }
    while (KEY3 == 0);          // ④ 等松手
}
```

机械按键闭合瞬间金属弹片会弹跳几毫秒，单片机跑得太快会把每次弹跳都当一次按下——表现为“按一次跳好几个数”。第 ④ 步是防止按住不放导致计数狂涨。

待优化：45ms 的消抖时间偏长（标准 10~20ms），快速连点会漏。另外 `while(KEY3==0)` 是阻塞的，第 3 步做定时器时要改掉。

(4) 数字转字符与两位数拆分

串口传的是字节，但你想显示的是字符，两者靠 **ASCII 码** 对应：

```
count + '0'           // 0~9 → '0'~'9' (即 +48)
count / 10 + '0'       // 十位：整除 10
count % 10 + '0'       // 个位：对 10 取余
'\n'                  // 换行 (ASCII 10)，本身就是字符，不需要再 +'0'
```

例：count = 27 → 27/10 = 2 → '2'；27%10 = 7 → '7'；连起来就是 "27"。

三、踩坑全记录 (15 条)

这一节是整份记录里最值钱的部分。**每条坑都实际发生过。**

【代码类】

坑 1：while(1) 后面的代码永远不会执行

现象：串口疯狂刷 A，一秒上千个，延时完全不起作用。

代码：

```
while(1)
{
    uart_send('A');
}
delay_ten_us(10000); // ← 这行永远执行不到
```

原因：while(1) 是死循环，程序进了就出不来，循环体后面的代码是死代码。

正解：延时要放在**循环体内部**。

坑 2：SBUF = count 发出去的不是字符

现象：串口助手几乎什么都看不到，偶尔莫名跳一行。

原因：发的是**数值**不是**字符**。count = 1 时 SBUF = 1，ASCII 1 是控制字符 SOH，屏幕不显示任何字形；count = 10 时发的是换行符；count = 13 是回车。

正解：SBUF = count + '0'；(+48，映射到 ASCII 的 48~57)。

坑 3：count 超过 9 会“跑出数字区” (ASCII 表实证)

现象：串口输出 123456789:;<=>?@ABCDEFGHIJKL。

原因：+ '0' 就是 + 48，只能把 0~9 映射到 '0'~'9' (48~57)。count 到 10 时 48+10 = 58，那是冒号：的编码；到 17 时 48+17 = 65，那是大写 A。等于沿着 ASCII 表一路往上爬，从数字区爬进标点区、再爬进大写字母区。

正解：两位数要**拆成十位和个位分别发送**（见 2.4 节）。

坑 4：调试时误信了串口助手的 HEX 模式

现象：看到 01 02 03 04 ... 79，以为数据全是错的。

原因：串口助手开着 **HEX 显示模式**，它把收到的每个字节用两位十六进制"原样"显示——收到数值 1 显示 01，**不是字符**。而 0x79 = 121 正好等于当时点击的次数，说明**串口传输本身完全正确**，一个字节都没丢。

正解：调试显示类问题时，先确认助手是 HEX 模式还是文本模式。**两种模式看到的東西完全不同。**

坑 5：函数定义与调用的写法混了

错误代码：

```
uart_send(u8 A);    // X
```

原因：把**定义**的写法和**调用**的写法混在一起了。

	写法	作用
定义	<code>void uart_send(u8 dat)</code>	告诉编译器参数 类型和名字
调用	<code>uart_send('A')</code>	只递 值 ，不写类型、不写名字

另外 A（无引号）是变量名，'A'（有引号）才是字符常量 65。

正解：`uart_send('A');` 或 `uart_send(count + '0');`

坑 6：'\n' 后面多加了 + '0'

错误代码：

```
uart_send(count\n + '0');    // X 两处错
```

原因：1. \n 是转义符，必须写成 '\n'（带引号，整体是一个字符），不能和变量拼在一起 2. '\n' **本身就是字符**（ASCII 10），不需要再转换

判断法则：

算出来的数字 → 要 + '0'；写好的字符 → 直接用。

如果加了 + '0'，10 + 48 = 58，屏幕上出现的是冒号而不是换行。

坑 7：消抖延时过长导致快速连点漏检

现象：消抖设成约 100ms 时，快速连点（每秒 5~6 下）会漏掉一些。

原因：消抖窗口太大，第二次按下可能正好落在"第一次还没确认完"的窗口里被吃掉。

正解：消抖时间取 **10~20ms** 即可。实际调试中改成了约 45ms，仍然偏长，可以再降。

【概念类】

坑 8：误以为 P0 口自带内部上拉

错误认知："P0 口不是自带上拉电阻吗"

事实：P0 是唯一没有内部上拉的口（P1 / P2 / P3 才有）。这正是"P0 必须外接排阻"这条规矩的来源——两件事是同一件事的两面。

端口	内部上拉	输出 1 时	要不要外接排阻
P0	没有（开漏）	悬空，电平不确定	必须接（当普通 IO 用时）
P1 / P2 / P3	有（弱上拉）	被托到高电平	不用

坑 9：把"上拉电阻"和"通信握手"混为一谈

错误认知："设备输入的时候不应该是先拉低然后建立通信，然后再传输数据吗"

澄清：上拉电阻不发起任何事情，它只是个"没人驱动时把引脚轻轻托在高电平"的被动元件，作用是防止引脚悬空（悬空电平会乱飘）。

"先拉低、再传数据"那是 UART 的起始位或 I2C 的 START 信号——属于协议层的约定，跟 IO 口里有没有上拉电阻是两个层级的事。而读一个按键根本不存在"建立通信"这一步，就是纯粹地"看一眼引脚现在是高还是低"。

坑 10：while(KEY3 == 0); 是阻塞的

现象：按住按键不放时，整个程序卡在这一行，其他事都干不了。

原因：while 空循环等松手，是"忙等待"。

现状：现在程序里没有其他任务，所以没暴露问题。

正解：第 3 步改成定时器 + 状态机后必须替换掉（用"边沿检测"代替"等松手"）。

【环境与工具类】

坑 11：PowerShell 直接调 git 会失败

现象：

```
git: 'remote-https' is not a git command
fatal: remote helper 'https' aborted session
```

原因：PortableGit 的 git.exe 在 cmd/ 目录下，而传输 helper（git-remote-https.exe）在 mingw64/bin/。PowerShell 直接调用时 PATH 里没有 mingw64/bin，git 找不到 helper。

正解：在 Git Bash 环境里调用 git（PATH 已配好）。若必须用 PowerShell，先补 PATH：

```
$env:PATH = "$base\mingw64\bin;" + $env:PATH
```

坑 12: PowerShell 里 git 中文全是乱码 (假象)

现象: `git log` 显示 `feat: 51 錦曠塤鏈轰覆鑒 i € 氫俊...`, `user.name` 显示 `璋紵騫?`。

原因: 编码解码问题, 不是数据问题。git 输出的是 UTF-8 字节, PowerShell 用系统默认的 GBK 去解码 → 显示乱码。用 UTF-8 读同一个文件 (谢伟平) 是正常的, 证明 **git 里存的数据完全正确**。

正解: 要看正确中文, 先设置:

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
```

坑 13: % 字符被安全策略拦截

现象: `git log --pretty=format:"%h"` 被执行策略拦住, 报 `"cmd.exe %VAR% environment variable syntax"`。

原因: `%h%n` 这种写法被安全策略误判成 `cmd` 的环境变量语法。

正解: 改用默认格式 `git log -1`, 或者不用 `%` 占位符。

坑 14: 中文路径下的 Edge 命令行会失败 (工具已知坑)

现象: 用 Edge 无头模式把 HTML 转 PDF 时, 如果传中文路径给 `--print-to-pdf=<路径>`, 会打印失败。

正解: 先写到 ASCII 临时路径, 成功后再复制到目标中文路径。(转 PDF 脚本已内置此规避。)

坑 15: 抽象图示看不懂——圆环图失败案例

现象: 用"8 位补码排成圆环"的图讲补码, 完全看不懂。

原因: 圆环是抽象的。对第一次接触补码的人来说, "为什么是圆的"这层就没建立起来, 直接看刻度更晕。

正解 (已验证有效): 1. **先给日常类比:** 钟表 12 小时制, $9+4=1$ 点; 1 点也可以读成 -11 点 2. **再换横向刻度尺:** 一条 0~255 的条, 上半贴无符号标签、下半贴有符号标签, 中间标分界线 3. **配具体数值对照表:**

00000001
= 1 / +1、10000000 = 128 / -128、11111011 = 251 / -5

通用原则: 看不懂抽象图时, 先换"日常类比 + 横向对照条 + 具体数值表", 别在抽象图上硬解释。

四、原理补课

这一节记录今天真正讲透的原理。**顺序很关键:** 先有感性经验 (摸过板子、跑过代码), 再补理论。

4.1 补码与溢出 (8 位)

从钟表讲起: 表盘只有 12 个刻度, 9 点过 4 小时是 1 点 ($9+4=13$, $13-12=1$)。表盘不关心过了几小时, 只认"除以 12 的余数"。同一个刻度 1, 也可以读成 -11 。

8 位二进制 = 模 256 的钟表, 刻度 0~255。人们约定把后半段 (128~255) 重新贴成负数标签:

补码里"负数 X"存的就是 $256 - X$ 这个正数。 $128 \equiv -128$ (因为 $128+128=256\equiv 0$)、 $255 \equiv -1$ ($255+1=256\equiv 0$)、 $251 \equiv -5$

两种读法对照：

二进制	无符号读法	有符号读法
0000 0001	1	+1
0111 1111	127	+127
1000 0000	128	-128
1000 0001	129	-127
1111 1011	251	-5
1111 1111	255	-1

为什么 -128 存在而 +128 不存在：256 个刻度里 0 占了一格，剩下 255 格分给正负数——255 是奇数，分不匀，于是负数区多拿一格拿到 -128，正数区只到 +127。

补码的真正目的：把减法变成加法，省掉减法器硬件。

$$5 - 3 = 5 + (-3) = 5 + 253 = 258 \equiv 2 \checkmark$$

两个溢出点，对应两个标志：

情况	无符号视角	有符号视角
$255 + 1 = 0$	超出 255， CY = 1	$(-1)+1=0$ 正确， OV = 0
$127 + 1 = 128$	$127+1=128$ 没问题， CY = 0	正数加正数得负数， OV = 1

CY 管无符号溢出，OV 管有符号溢出。 硬件实现： $OV = C_6 \oplus C_7$ （次高位进位异或最高位进位）。

验证实验：char a = 127; a = a + 1; → a 变成 -128。在 Keil 里单步看 PSW 的 OV 位翻转。

4.2 ASCII 码表

串口线只能传字节（0~255），但人要看的是符号（字母、数字、换行）。哪个数字代表哪个符号的约定表就是 ASCII 码表（1963 年制定，比 51 还老）。

分段结构：

范围	内容
0 ~ 31	控制字符（不可显示）：8=退格、9=Tab、10=换行、13=回车
32	空格
32 ~ 47	标点 !"#\$%&'()*+,-./
48 ~ 57	数字 0 ~ 9

范围	内容
58 ~ 64	标点 : ; <=> ? @
65 ~ 90	大写 A ~ Z
91 ~ 96	标点
97 ~ 122	小写 a ~ z

必记的值：'0' =48、'9' =57、'A' =65、'a' =97、空格=32、换行(\n)=10、回车(\r)=13。

漂亮规律：'A' =65、'a' =97，正好差 32（二进制第 5 位）。所以大小写转换在机器里只是改一个比特。

在 C 语言里：'A' 和 65 是同一个东西，编译器自动转换。所以 count + '0' 实际就是 count + 48。

4.3 51 的九个"为什么"

① char 为什么是 -128~127，OV 怎么来的

见 4.1。核心：256 个编码里 0 占一格，负数多拿一个。

② ALE 的频率为什么是 fosc/6

一个机器周期 = 6 个状态（S1~S6），硬件规定 ALE 在 S1P2 和 S4P2 各发出一次有效脉冲 → 每机器周期 2 次 → $f_{osc}/12 \times 2 = f_{osc}/6$ 。12MHz 时就是 2MHz。

ALE 全名 Address Latch Enable，用于把 P0 口复用的低 8 位地址锁进外部锁存器（74HC373）。这个引脚的存在本身就说明 51 是为"外部扩展"而设计的。

③ ACC、PSW、DPTR 为什么长这样

51 是累加器型 + 单总线结构（1970 年代晶体管贵的产物）：

- **ACC**：ALU 的一个输入固定来自 ACC，结果也必须回 ACC → 所以只能写 `ADD A, R0`，不能写 `ADD R0, R1`。对比 ARM 的 `ADD R0, R1, R2`（寄存器-寄存器型）
- **PSW**：CY（进位，兼位操作的位累加器）、AC（半进位，给 DA A 做 BCD 调整）、OV（有符号溢出）、P（奇偶，串口校验用）、RS0/RS1（切换 4 组工作寄存器——中断时不用压栈，直接切一组，是 51 的快速上下文切换技巧）
- **DPTR**：16 位，DPH + DPL 拼出来。CPU 是 8 位但要访问 64KB 空间，所以必须有 16 位地址寄存器，它是唯一能直接给 16 位地址的

④ MOV / MOVX / MOVC 的区别

因为 51 是哈佛结构（程序空间和数据空间分开，地址编号可以重复）：

指令	访问哪里	硬件信号
MOV	片内 RAM / SFR（8 位地址）	无外部动作
MOVX	片外数据存储器（X = eXternal，16 位地址）	拉低 RD 或 WR
MOVC	程序存储器（C = Code，只读）	拉低 PSEN

物理证据：拿逻辑分析仪量，MOVC 拉 PSEN、MOVX 拉 RD/WR，两者不会混。

这解释了 C51 里的关键字：code（放 Flash，用 MOVC 读，因为 51 只有 128~256 字节 RAM）、xdata（外部 RAM，用 MOVX）。

⑤ 中断向量表为什么从 0003H 开始、每隔 8 字节

地址	用途
0000H	复位入口（只有 3 字节，正好放一条 LJMP main）
0003H	外部中断 0
000BH	定时器 0
0013H	外部中断 1
001BH	定时器 1
0023H	串口
0030H 起	主程序区

为什么从 0003H 开始：0000~0002 被复位入口占了（刚好一条 LJMP 的长度）。**为什么隔 8 字节：**8 字节够放一条长跳转指令，中断发生时硬件自动把 PC 指向这个固定地址——这叫**向量中断**（硬件直接给入口，不用软件查表）。

C 语言里 void timer0() interrupt 1 的 1 **就是中断号**，编译器据此把跳转放到 000BH。

⑥ 程序为什么从 0000H 开始、为什么必须 while(1)

复位时硬件把 PC 强行置成 0000H（芯片设计时焊死的电路）。而 0000H~0002H 只有 3 字节，恰好够放一条 LJMP main。

为什么必须 while(1)：PC 的行为只有一个——**取完一条指令就自动 +1，永不停息**。51 上没有操作系统，不存在“程序结束”。main 最后一条指令跑完，PC 继续往下读 Flash 空白区（内容是 0xFF，对应 MOV R7,A），一路瞎跑 → LED 乱闪、莫名复位。

⑦ P0 为什么加上拉、为什么读引脚前要先写 1

P0 为什么必须外接上拉：它的输出级**只有一个下拉 NMOS，没有上拉**。写 0 时管子导通把引脚拉低；写 1 时管子截止——**引脚悬空**，电压不确定。这叫**开漏输出**。做成开漏是因为 P0 兼做地址/数据复用总线 AD0~AD7，总线上开漏可实现“线与”，且驱动能力可由外部电阻调整。

P1/P2/P3 有内部弱上拉（等效几十 kΩ），所以不用外接。

为什么读引脚前要先写 1：如果上次输出过 0，那个下拉管**还导通着**，直接把引脚钳在低电平——外部按键拉高也没用，**读进来永远是 0，你完全分辨不出按没按**。先写 1 就是让下拉管截止、“松开”引脚。

关键提醒：51 复位后所有 IO 口锁存器默认是 0xFF（已写 1 状态），所以上电直接读是安全的。只在“先输出过 0、再读输入”的顺序下才会爆炸——这也是很多教程不提这个坑的原因。

为什么叫“准双向口”：真正的双向口有独立的方向寄存器（如 STM32 的 MODER，配成输入时输出驱动器在硬件上断开）。51 为了省晶体管，把输出驱动和输入缓冲**永久并联**在同一根引脚上，代价就是这个坑。

顺带的实用结论：51 的 IO 口**灌电流 (sink) 能力强、拉电流 (source) 能力弱**（弱上拉的推力太小）。所以开发板上的 LED **几乎全是低电平点亮**——LED 一端接 VCC，单片机输出 0 时电流灌进引脚。反接用高电平点亮会明显变暗。

⑧ 定时器初值怎么算、方式 1 和方式 2 的区别

定时器本质 = 对机器周期计数的加计数器。12MHz 下每 1 μ s 加 1，所以要反着算初值：

计数个数 N = 想要的微秒数 (12MHz 下)

初值 = 65536 - N ← 方式 1 (16 位)

TH0 = 初值 / 256; TL0 = 初值 % 256

例: 1ms $\rightarrow N = 1000 \rightarrow$ 初值 = 64536 = 0xFC18 $\rightarrow$ TH0 = 0xFC, TL0 = 0x18

方式 2 (8 位自动重装) 为什么重要: 方式 1 溢出后计数器归零，必须在中断里手动重装，中间隔着中断响应延迟 (3~8 个机器周期) + 重装指令时间 $\rightarrow$ 溢出率抖动。方式 2 是 TL 计数、TH 存重装值，溢出瞬间硬件自动把 TH 拷回 TL，零延迟零抖动。

这就是串口波特率必须用 T1 方式 2 的原因。

⑨ 波特率为什么用 11.0592MHz

见 2.4 节。核心: 分母 384 = 12 $\times$ 32 固定, f_{osc} 要能被 384 $\times$ 波特率 整除。11.0592MHz 整除 4800/9600/19200; 12MHz 得 10417, 误差 8.5% $\rightarrow$ 乱码。

4.4 矩阵键盘扫描原理

4 行 $\times$ 4 列共 16 个按键，每个按键两端分别接一条行线和一条列线。

	C1	C2	C3	C4	
R1	S1	S2	S3	S4	R1 输出 0
R2	S5	S6	S7	S8	R2 输出 1
R3	S9	S10	S11	S12	R3 输出 1
R4	S13	S14	S15	S16	R4 输出 1
读列:	1	0	1	1	

扫描逻辑: 逐行拉低 (其余行输出 1) $\rightarrow$ 读 4 条列线 $\rightarrow$ 哪一列为 0，按键就在 (当前行，那一列)。

上例中 R1 拉低时 C2 读到 0 $\rightarrow$ 按键 = S2。

按键编号换算: (行号 - 1) $\times$ 4 + 列号

这一节最容易踩的坑: 列线当输入读之前必须先写 1 (就是 4.3 ⑦ 那个坑在矩阵键盘上的原样重现)。不写 1 的话列线永远是 0，程序会认为每一行的每一个按键都被按着。

另一个坑: P1 口同时装着行和列，改某一位时**不能用** P1 = 0xEF **整体赋值** (会覆盖列线的"写 1"状态)，要用位操作或位寻址。

待办: 等有万用表 (或会用串口反查引脚) 后，把这个矩阵键盘版本做出来。

五、学习方法：输出型打法

前提认知：输出型不是缺点，有认知科学依据——**生成效应**（自己生成的内容记忆强度远高于阅读）和**测试效应**（主动回忆一次 > 被动重读三次）。觉得背书痛苦，是因为背诵是**最低效的输入方式**。

方式	怎么做	效果
① 破坏性实验	学一个原理，立刻改一个参数让它崩掉，观察现象	最高
② 讲给别人听	讲给完全不懂的人听（或讲给 AI，让它挑刺）	很高
③ 写成公开教程	发到自己的文档站	高
④ 做题 / 默写	关上书，把图或公式默写一遍	中
✗ 反复重读划线	—	最低

今天的实际验证：所有记住的东西，都是"自己写出来"或"自己撞出来的"——

- 串口初始化 5 个寄存器，一次写对（因为理解每个寄存器在干什么）
- 15 个坑，全部是从实际报错里长出来的
- ASCII 码表，因为亲眼看到 123456789:;<=>?@ABC 才真正记住

具体执行框架：

看 5 分钟 → 关上书 → 把图/公式默写一遍 → 做 2 道计算题 → 找一个能动手验证的点

进阶用法：把学习记录写成公开教程。**一份时间三份收益**——既巩固记忆，又是作品集素材（简历三根柱子之一），还练了技术写作。

六、水平定位与路径

51 能力四级阶梯

级别	名称	标志能力
L1	能点灯	最小系统、GPIO、软件延时、LED/按键/蜂鸣器
L2	会用外设	定时器中断、外部中断、串口、数码管扫描、PWM
L3	能做项目	非阻塞状态机、I2C/SPI 传感器、独立排查问题 ← 目标
L4	能迁移	读 datasheet 配寄存器、volatile/原子性、看门狗/低功耗、能换到 STM32

实测定位：L2 偏下

三道实测题的表现：

题目	考察	表现
P1=0x00 后读按键	L1→L2	✅ 现象答对（读到恒 0、灯会亮），但没意识到"灯一直亮 = 按键失效"
定时器非阻塞程序	L2→L3	思路方向对（用定时器计数、要消抖）， 但没写过

题目	考察	表现
volatile + 原子性	L3→L4	没概念

维度画像：

维度	状态
GPIO / 准双向口	原理已通
外部中断（按键中断）	用过，且踩过坑
"中断为什么不能加延时"	真懂（亲眼见过后果）
数码管动态扫描	做过，原理待验
定时器中断 / 非阻塞设计	空白 ← 最大短板
volatile / 原子性	没概念
串口	✅ 今天补上了
I2C / SPI / 调试工具	未测

关键判断

目标是 L3，不是 L4。

简历上要的是"我独立做过什么"（作品集），L3 就够产出这个。L4 是面试聊深度用的，应该在**做项目的过程中顺带长出来**，专门去补性价比很低。

别为了"学完"而学。

七、工具与环境备忘

7.1 git 环境要点（踩过坑）

项目	说明
git 位置	C:\Users\23147\.workbuddy\binaries\PortableGit\versions\1.2.0\
必须用 Git Bash 调用	PowerShell 直接调 cmd\git.exe 会缺 remote-https helper
凭据	credential.helper = manager (GCM)，已配好，push 不用再输密码
代理	.gitconfig 里 http.proxy / https.proxy = http://127.0.0.1:7897 (Clash)
中文显示	PowerShell 里看 git 输出是乱码属正常（GBK 解码问题），数据本身是好的

7.2 仓库清单

仓库	本地路径	远程状态
51 学习	D:\51单片机工程	✅ github.com/huhuhuhu5555/51-.git (Public)
campus-auth	D:\campus-auth	❌ 无 remote。若要上传必须用 Private

仓库	本地路径	远程状态
WiFi	D:\WiFi	✗ 无 remote。若要上传必须用 Private

关于 campus-auth / WiFi 的结论：这两个**不建议上传 GitHub**（即使私有）。原因：它们是校园网认证绕过工具和流量伪装工具，上传到第三方平台会成倍扩大暴露面，而且**没有任何收益**（不是作品集、不能展示、备份也可以本地做）。备份用**加密压缩包 + 移动硬盘**更安全。

如果目的是丰富作品集：**代码不能公开，但经验可以**——把技术原理和踩坑过程写成文章（不附代码、不提供工具）是安全且加分的。

7.3 建议购买的工具

工具	价格	为什么值得
USB 逻辑分析仪	30~50 元	学 DS18B20 / I2C / SPI 时 能看见真实时序波形 。从"看结果对不对"升级到"看时序对不对"，是调试能力的起点
万用表	几十元	量电压、查短路、测通断。P0 上拉那个坑有万用表能 10 秒定位

八、下一步计划

立即

- [] 去掉两位数显示的**前导零**（count < 10 时不发十位，用 if 判断十位是否为 0）
- [] 把消抖时间从 45ms 降到 **10~20ms**，验证快速连点不漏

第 2 步：DS18B20 温度采集

用**已学过的**模块做真项目，全程**不翻教程**：

- 用：DS18B20（单总线）+ 串口
- 练：**严格时序**（复位脉冲 480μs、读写时隙都是微秒级）、数据格式化
- 产出：电脑上实时看温度
- 验证方式：手捏传感器，温度数字应该上涨

第 3 步：定时器中断做周期采样

- 用：定时器中断
- 练：**非阻塞设计**——中断里只做"置标志/存数据"，主循环做决策
- 产出：把第 2 步的"每秒采一次"改成非阻塞，**顺带做掉那道没做出来的实测题**
- 关键：把 while(KEY3==0) 这种阻塞等待全部替换掉

之后的关卡（预判会遇到）

1. **两位以上数字的格式化输出**（现在只支持 0~99）
2. **volatile 为什么必须加**——编译器看不到中断的修改，会把主循环的读取优化掉
3. **16 位变量在 8 位机上的原子性问题**——中断可能打断到一半，主循环读到"撕裂值"

这三个坑不用现在去学，**写到那一步自然会撞上**，撞上再理解最牢。

附录：知识点速查

项目	值 / 结论
12MHz 机器周期	1 μ s (12 个时钟 = 6 个状态)
11.0592MHz 机器周期	约 1.085 μ s
9600 波特率初值 (11.0592MHz)	TH1 = TL1 = 0xFD
波特率公式	fosc $\div$ (384 $\times$ (256 - TH1))
ALE 频率	fosc $\div$ 6 (12MHz $\rightarrow$ 2MHz)
'0' / '9'	48 / 57
'A' / 'a'	65 / 97 (差 32)
换行 \n / 回车 \r	10 / 13
数字转字符	数字 + '0'
两位数拆分	十位 n/10 , 个位 n%10
8 位补码范围	-128 ~ +127
补码负数 X 的机器表示	256 - X
溢出标志	CY 管无符号, OV 管有符号
定时器 1ms 初值 (12MHz)	0xFC18 (TH0=0xFC, TL0=0x18)
P0 口	无 内部上拉, 必须外接
P1/P2/P3	有弱上拉, 不用外接
串口引脚	TXD = P3.1, RXD = P3.0
读 IO 前	必须先写 1 (准双向口)
按键接法	一端接 IO、一端接 GND, 读到 0 = 按下
消抖时间	10 ~ 20 ms

记录于 2026-10-01